

DAY 1

Module 1

From Vibe Coding to Spec-Driven Development

Why unstructured AI prompting breaks down, and what spec-driven development changes.

What You'll Learn

- 1 Recognize the failure modes of unstructured AI prompting
- 2 Explain the core thesis of spec-driven development
- 3 Locate GitHub Spec Kit and OpenSpec within the wider SDD landscape
- 4 Scaffold both frameworks side by side in your own project

THE PROBLEM

Failure Modes of Ad Hoc Prompting

Context drift: the agent gradually loses track of what it's building as a conversation grows longer.

Hallucinated APIs: confident code written against methods, endpoints, or libraries that don't actually exist.

Unreviewable diffs: large, sprawling changes with no clear boundary to review against.

Silent scope creep: the agent quietly does more, or less, than what was actually asked.

The Core Thesis of Spec-Driven Development

The specification, not the code, is the source artifact.

Code becomes a build output generated from that source.

This flips the usual relationship: instead of code as truth and docs as commentary, the spec is truth and code is compiled from it.

Because the spec persists and stays current, it doubles as living documentation and an audit trail.

Multiple frameworks implement the same core thesis, differently.

GitHub Spec Kit, OpenSpec, AWS Kiro, BMAD-METHOD, and Tessel all put a written spec ahead of code.

This course teaches two hands-on: Spec Kit as the primary framework, OpenSpec for comparison.

Neither framework requires a specific AI coding agent: both work with GitHub Copilot, Claude Code, and others.

APPLYING THIS WITH

GitHub Spec Kit

Scaffolding a Project

```
specify init
```

Scaffolds a new project: creates .specify/ with templates, scripts, and agent-specific command files. Use --integration copilot or --integration claude to target your agent.

The map: The full command chain this course follows: constitution → specify → clarify → plan → checklist → tasks → analyze → implement → converge. This is your map for the next eight modules.

Not all mandatory: Only specify is strictly required before plan. Clarify, checklist, and analyze are quality gates you add for anything with meaningful ambiguity.

APPLYING THIS WITH

OpenSpec

Scaffolding a Project

```
openspec init
```

Scaffolds openspec/ with specs/ (source of truth), changes/ (proposed updates), and config.yaml. Install first with `npm install -g @fission-ai/openspec`.

The default loop: Default quick path: explore (optional) → propose → apply → sync (optional) → archive.

Different spec model: Where Spec Kit produces one spec file per feature, OpenSpec proposes changes as compact deltas (ADDED, MODIFIED, REMOVED) against a single living spec.

Module 1 at a Glance

	GitHub Spec Kit	OpenSpec
Install	<code>specify init --integration <agent></code>	<code>npm i -g @fission-ai/openspec + openspec init</code>
Creates	<code>.specify/</code> (templates, scripts, agent files)	<code>openspec/</code> (specs/, changes/, config.yaml)
Spec model	One file per feature	Single living spec + deltas

Key Takeaways

- Ad hoc prompting fails in predictable, nameable ways.
- SDD's core move: make the spec the source of truth, not an afterthought.
- You now have both frameworks scaffolded and ready to use for the rest of the course.

HANDS-ON NEXT

Form your project team, receive your team's assigned requirement, and set up your agentic coding tool against it. Scaffold both a Spec Kit specs/ directory and an OpenSpec openspec/ directory, then run a basic and an advanced prompt to compare direct prompting against structured, spec-driven input.

Next: Module 2: Discovery at Scale: Understanding an Existing Hierarchical Codebase